# DSC 40B - Discussion 01

**Problem 1.**

What is the time complexity of the following functions? State your answer using $\Theta$ notation.

**a)**
```python
def foo(n):
    for i in range(n**2 - 2*n + 100):
        j = 0
        while j < n:
            j += 1
```

**b)**
```python
def foo(n):
    while n > 1:
        n /= 10
        print(n)
```

**c)**
```python
def foo(n):
    for i in range(n):
        for j in range(i**2): # <-- notice the bound!
            print(i + j)
```

**Problem 2.**

Let $f(n) = \sum_{p=0}^{n} 3^p$. What is $f$ in $\Theta$ notation?

**Problem 3.**

Consider the algorithm below.

```python
def bogosearch(numbers, target):
    """search by randomly guessing. `numbers` is an array of n numbers"""
    n = len(numbers)

    while True:
        # randomly choose a number between 0 and n-1 in constant time
        guess = np.random.randint(n)
        if numbers[guess] == target:
            return guess
```

We will set up the analysis of the expected time complexity of this algorithm.

**a)** What are the cases? How many are there?

**b)** What is the probability of case $\alpha$?

**c)** What is the running time in case $\alpha$?

**Problem 4.**

For each problem below, state the largest theoretical lower bound that you can think of and justify (that is, your bound should be "tight"). Provide justification for this lower bound. You do not need to find an algorithm that satisfies this lower bound.

*Example*: Given an array of size $n$ and a target $t$, determine the index of $t$ in the array.

*Example Solution*: $\Omega(n)$, because in the worst case any algorithm must look through all $n$ numbers to verify that the target is not one of them, taking $\Omega(n)$ time.

**a)** Given an array of $n$ numbers, find the second largest number.

**b)** Given an array of $n$ numbers, check to see if there are any duplicates.

**c)** Given an array of $n$ integers (with $n \geq 2$), check to see if there is a pair of elements that add to be an even number.

**Problem 5.**

Consider the code below where heights is an array of n elements:

```
for i in range(n):
        for j in range(2*i):
                height = heights[i] + heights[j]
```

What is the time complexity of the code?